6. Etapa 1: Preparar el caso (Casos de Uso Seleccionados)

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Solicitar cotización inicial (US-01, US-02, US-03)	Cliente / Coordinador	1 persona y 1 hora base por cada 50 m². Añadir tiempo y tarifa de extras.	Precio y duración estimada calculados automáticamente, solicitud guardada.
CU-02	Asignación y programación de cuadrilla (US-04, US-05, US-06)	Coordinador	Límite máximo e inamovible de 8 horas diarias de trabajo por cuadrilla. Sugerencia por cercanía.	Proveedor (cuadrilla) vinculado a la solicitud y servicio programado.
CU-03	Registro de cierre y evidencias en campo (US-07, US-08, US-09)	Supervisor	Exigir hora, geolocalización, checklist de áreas marcado al 100% y firma digital del cliente.	Servicio cambia a estado finalizado, detona cobro / pre-factura.
CU-04	Recálculo de precio por excedente (US-14)	Coordinador / Sistema	Sincronización financiera en tiempo real. Ajuste de tarifa si la superficie real es mayor a la reportada.	Importe actualizado, cliente notificado y estado de cuenta sincronizado.

7. Etapa 2: Elegir el patrón de lógica de negocio (Tabla comparativa)

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Agrupa la lógica directamente en una función simple. Ideal para un prototipo rápido (PIA).	Vincula la lógica directamente a las tablas de la BD, causando rigidez si cambian.	Encapsula comportamiento en objetos de dominio. Genera sobreingeniería para este alcance.	BRD: Cálculo dinámico de 1 persona / 1 hora por cada 50 m².
Cambios esperados	Se adapta fácil al inicio, aunque a nivel empresarial a largo plazo podría generar código espagueti.	Complicado de mantener si la estructura de la base de datos de PostgreSQL/Supabase cambia.	Altamente escalable, pero exige un diseño inicial y mapeo de datos que excede el tiempo del proyecto.	Alcance del proyecto (PIA universitario) centrado en resolver el prototipo funcional actual.
Interacción entre conceptos	Accede directamente a los datos necesarios para resolver un caso de uso cruzando la información.	Obliga a manejar la lógica dividida rígidamente por cada tabla relacional.	Comunica Entidades y Value Objects a través de Eventos de Dominio.	US-08 y 09: Para cerrar el servicio interactúan el checklist, la firma y la ubicación.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Estados e invariantes	Las validaciones se hacen mediante condicionales procedimentales antes de guardar.	La validación depende estrictamente del esquema de datos.	El Agregado Raíz protege los invariantes internamente antes de emitir cambios.	RN-02: Bloqueo automático para evitar que se asignen más de 8 horas.
Velocidad de desarrollo	Muy rápida. Se alinea perfectamente con backends modernos en JS/Node interactuando directo con la BD.	Moderada, requiere crear controladores por tabla.	Lenta, exige capas de mapeo (ORM) e interfaces robustas para aislar el dominio.	Limitante temporal: es una entrega académica de arquitectura modular.

9. Etapa 4: Diseñar la Service Layer

Servicio	Método o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
SolicitudesAppService	crearCotizacion	datosInmueble, serviciosAdicionales	Repositorio de Solicitud y Cotización	ID de Solicitud	Delega al Transaction Script calcular tarifa según reglas de 50m².

Servicio	Método o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
AsignacionAppService	programarServicio	idSolicitud, idProveedor	Repositorio de Proveedores (Cuadrillas)	Confirmación de agenda	Ejecuta verificación del límite de 8 horas de trabajo diario.
EjecucionAppService	registrarCierre	idServicio, checklist, firmaDigital	Repositorio de Servicios	Estado "Finalizado"	Valida captura obligatoria del checklist al 100% y firma del cliente.
FinanzasAppService	recalcularPorExcedente	idServicio, nuevosMetros	Repositorio de Cargos y Administración	Nuevo importe	Asegura sincronización financiera en tiempo real para evitar pérdidas.

10. Etapa 5: Separar persistencia y transacciones

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
ISolicitudRepository	Almacenar cotizaciones e historial de peticiones del cliente.	guardarSolicitud(solicitud), obtenerPorId(id)	Agregado: SolicitudServicio
IProveedorRepository	Conocer el personal de	obtenerDisponiblesPorFecha(fecha)	Entidad: Proveedor

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
	limpieza disponible y su agenda.		(Técnico / Cuadrilla)
IServicioRepository	Actualizar estados de limpieza, checklist y capturar incidencias/firmas.	actualizarEstado(servicio), guardarEvidencia(id, datos)	Entidad: Servicio

11. Etapa 6: Separar Commands y Queries (CQRS Básico)

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Command	ProgramarServicioCommand	idSolicitud, idProveedor, fecha	ProgramarServicioHandler	void / success	Modifica el estado del sistema vinculando una cuadrilla a una solicitud pendiente.
Command	CerrarServicioCommand	idServicio, checklist, firma	CerrarServicioHandler	void / success	Cambia fuertemente el estado a "Finalizado" e inserta registros de evidencia.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Query	ObtenerCuadrillasSugeridasQuery	fecha, idSolicitud, ubicacion	ObtenerCuadrillasHandler	Lista	Lectura compleja para el algoritmo de sugerencias (ubicación/horario) sin alterar datos.
Query	ConsultarAgendaCoordinadorQuery	fechaInicio, fechaFin	ConsultarAgendaHandler	Lista	Visualización masiva de datos en el portal web (US-05) que solo lee información.

14. Bitácora obligatoria de IA

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	Elegir el patrón de arquitectura de lógica de negocio.	"Compara TS, TM y DM basado en las reglas del BRD de CleanPro."	Propuso el uso inicial de Domain Model por la complejidad de validaciones.	Asumía que el sistema debía escalar a largo plazo corporativamente.	Optar por Transaction Script, justificando que es un proyecto académico (PIA) ágil.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
2	Diseñar la Service Layer (Application Services).	"Diseña los Application Services orquestando el mapa de dominio de CleanPro."	Creó métodos limpios que coordinan peticiones delegando reglas a la BD o TS.	Omitió cubrir al principio la totalidad de las 15 Historias de Usuario exactas.	Refinar los métodos para que cubrieran puntualmente desde la US-01 hasta la US-15 sin excepciones.

15. Etapa 9: Validar la propuesta

Validación	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Sí. Las capas de aplicación y repositorios cubren desde la US-01 (Cotización) hasta la US-15 (Reprogramación).
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Sí. Se normalizó el uso de "SolicitudServicio", "Proveedor" (Cuadrilla), y "Checklist".
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	Sí. Debido a la naturaleza de PIA, las reglas residen en el Script de Transacción y los repositorios, evadiendo sobreingeniería.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Sí. Los actores (Cliente, Coordinador, Supervisor) y

Validación	Pregunta	Resultado del equipo
		servicios cuadran con el ADR-001 (Arquitectura modular).
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Sí. Todo acceso a datos pasa a través de las interfaces ISolicitudRepository e IServicioRepository.
Transacción	¿El límite de la operación y su manejo de error están definidos?	Sí. Mediante CQRS se delimitó qué métodos solo devuelven datos y cuáles ejecutan transacciones atómicas.
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Sí. Documentado formalmente en la tabla de bitácora y adaptado al alcance universitario.